

A build-ready reference for the engineering agent: architecture, roles, data model, APIs, UI system, integrations, security and implementation sequence — reorganized into scannable tables and short call-outs for fast reading.

Font: Noto Sans • Radius 10–14px • 4/8px spacing rhythm

# 01 What Antigravity Must Build

*One continuous platform, not a stack of disconnected dashboards.*

A secure, scalable, AI-enabled national platform for land-governance research and policy — repository, AI discovery, collaborative workspaces, GIS, analytics, policy simulation, an innovation portal and public APIs, all built as one modular web app.

Engineering principle: one gateway → authenticated request → core service → internal data / live external source → response aggregation → UI.

## MVP priority order

| Pri. | Capability         | MVP definition                                                             |
|------|--------------------|----------------------------------------------------------------------------|
| P0   | Identity + RBAC    | 5 application roles, protected routes, permission middleware, audit events |
| P0   | Repository         | Upload, metadata, document preview, tags, provenance, access control       |
| P0   | AI Search          | Keyword + semantic search, filters, citations, related resources           |
| P0   | GIS                | Map viewer, public layers, land-use visualization, PostGIS queries         |
| P0   | Dashboards         | Research, policy, land-use, dispute and project indicators                 |
| P1   | Research Workspace | Collections, collaboration, saved searches, notes                          |
| P1   | Policy Simulator   | Scenario inputs, model run, explainable output, comparison                 |
| P1   | Innovation Portal  | Challenges/grants, submissions, review workflow                            |
| P1   | Grievance Tracker  | Geo-tagged land-dispute submission + status/triage for authorized roles    |

## 02 Target Architecture

*Same request flow as the source blueprint, adapted to Node.js + Express + PostgreSQL-first storage.*

| Layer                                                                                                                                                                                                                                          | Components                                                                                                           |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|----------------------------------------------------------------------------------------------------------------------|
| Client                                                                                                                                                                                                                                         | React — responsive web                                                                                               |
| API Gateway                                                                                                                                                                                                                                    | Express.js / REST                                                                                                    |
| Auth + RBAC                                                                                                                                                                                                                                    | JWT / role checks                                                                                                    |
| Core services                                                                                                                                                                                                                                  | Search (AI/semantic) • GIS (PostGIS/maps) • Analytics (Pandas/Spark) • Simulator (what-if) • Repository (docs/cases) |
| Data layer                                                                                                                                                                                                                                     | PostgreSQL + PostGIS • Elasticsearch index • Object storage (documents) • Redis / queue (cache + jobs)               |
| External APIs                                                                                                                                                                                                                                  | Bhuvan • OSM • Sentinel • data.gov.in                                                                                |
| Important: PostgreSQL is the system of record. PostGIS handles spatial data. Elasticsearch is a derived search/index layer — never the authoritative database. Object storage holds original files; PostgreSQL holds metadata and permissions. |                                                                                                                      |

### Request lifecycle

- Client → Gateway: browser/mobile sends an HTTPS REST request.
- Gateway → Auth: validate the JWT/session and attach user + role claims.
- Auth → Core service: route by capability; enforce permission before data access.
- Core service → data: query PostgreSQL/PostGIS, Elasticsearch, object storage or Redis.
- Core service → external API: call a live source only when cached internal data is absent/stale and policy permits.
- Response: normalize service outputs into one consistent API envelope, return through the gateway.

## 03 Technology Stack & Boundaries

*PostgreSQL is selected as the sole primary relational database for this build — MongoDB is not required for the MVP.*

| Layer           | Selected stack          | Rule for Antigravity                                                          |
|-----------------|-------------------------|-------------------------------------------------------------------------------|
| Frontend        | React + Vite + Tailwind | Implementation choice — source doesn't prescribe a framework                  |
| UI generation   | Stitch MCP              | Screen generation & iteration, then implement stable components in code       |
| Backend runtime | Node.js                 | Single backend runtime for all API services                                   |
| API framework   | Express.js              | REST-first gateway, modular route/controller/service structure                |
| Auth            | JWT + RBAC              | Short-lived access token; role/permission middleware; audit sensitive actions |
| Primary DB      | PostgreSQL              | System of record — users, metadata, workflows, policy runs                    |
| GIS DB          | PostGIS                 | PostgreSQL extension — geometry, spatial filtering, GeoJSON, map queries      |
| Search          | Elasticsearch           | Keyword, faceted and semantic/hybrid search index                             |
| Documents       | Object storage          | Original files; DB stores metadata, checksum, ownership, object key           |
| Cache / jobs    | Redis + BullMQ          | Fast cache and background jobs in Node                                        |

| Layer     | Selected stack                                  | Rule for AntigraVity                                                                |
|-----------|-------------------------------------------------|-------------------------------------------------------------------------------------|
| Analytics | Pandas/NumPy (+Spark)                           | Python analytics workers for heavy jobs; results exposed to Node internally         |
| AI/NLP    | Sentence-Transformers / spaCy; LLM API optional | Embeddings, entity extraction, summarization, RAG — keep provider adapters isolated |

## 04 Roles & Permission Model

*Five clean tiers. GIS experts fold into Researcher; NGOs are treated as elevated public users.*

| Role               | Access summary                                                              |
|--------------------|-----------------------------------------------------------------------------|
| Public / Citizen   | Search • AI assistant • public GIS • dashboards • grievance submission      |
| Researcher         | Public + upload • RAG • workspaces • GIS analysis • policy drafts           |
| Institution        | Researcher + org roster • bulk upload • shared-workspace oversight          |
| Policymaker / Govt | Analytics • simulation • dispute triage • reports • innovation-portal admin |
| Admin              | Users/roles • API keys • moderation • audit logs                            |

### Permission design

- Role → permission: store granular permissions (e.g. `repository.upload`, `gis.layer.create`) and map them to roles.
- Resource ownership: users access their own resources plus explicitly shared workspace resources.
- Institution boundary: institution managers manage only their own organization's researchers/workspaces.
- Government boundary: policymaker/government permissions unlock decision-support and official reporting.
- Admin boundary: platform administration is kept separate from business-domain permissions.

### Permission catalogue

|                               |                                |                                 |
|-------------------------------|--------------------------------|---------------------------------|
| <code>repository.read</code>  | <code>repository.upload</code> | <code>repository.publish</code> |
| <code>search.use</code>       | <code>gis.view</code>          | <code>gis.create</code>         |
| <code>analytics.view</code>   | <code>simulator.run</code>     | <code>workspace.create</code>   |
| <code>workspace.manage</code> | <code>innovation.submit</code> | <code>innovation.review</code>  |
| <code>grievance.submit</code> | <code>grievance.triage</code>  | <code>reports.generate</code>   |
| <code>audit.view</code>       | <code>users.manage</code>      | <code>api_keys.manage</code>    |

## 05 PostgreSQL + PostGIS Data Blueprint

*Normalized and auditable: UUID primary keys, created\_at/updated\_at, soft-delete, provenance on imports.*

| Domain                                                                                                                                                                                      | Core tables                                                     | Key fields                                                            |
|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------|-----------------------------------------------------------------------|
| Identity                                                                                                                                                                                    | users, roles, permissions, user_roles                           | email, name, org_id, status, role mapping                             |
| Institutions                                                                                                                                                                                | institutions, institution_members                               | name, type, verification status, member role                          |
| Repository                                                                                                                                                                                  | documents, document_versions, document_tags, sources            | title, abstract, file_key, checksum, source, visibility, published_at |
| Research                                                                                                                                                                                    | projects, workspaces, workspace_members, notes                  | owner, collaborators, status, resources                               |
| GIS                                                                                                                                                                                         | spatial_layers, spatial_features, map_projects                  | layer_type, geometry, CRS/SRID, source, temporal range                |
| Analytics                                                                                                                                                                                   | datasets, indicators, analytics_runs, metric_results            | dataset, metric, period, geography, result JSON                       |
| Simulation                                                                                                                                                                                  | simulation_models, simulation_runs, scenarios, scenario_outputs | model, inputs JSON, output JSON, explainability JSON                  |
| Disputes                                                                                                                                                                                    | grievances, grievance_events, locations                         | category, status, geometry, priority, assigned_to                     |
| Innovation                                                                                                                                                                                  | challenges, submissions, reviews                                | challenge, applicant, stage, score, review                            |
| Audit                                                                                                                                                                                       | audit_logs, api_keys                                            | actor, action, resource, IP hash/metadata, timestamp                  |
| GIS rules: use PostGIS geometry/geography types with explicit SRID; expose GeoJSON at the API boundary; never store raw map layers as unstructured blobs when spatial querying is required. |                                                                 |                                                                       |

## 06 Express.js API Blueprint

*Build as a modular monolith first — split into services later only if scale requires it.*

| Module     | Example endpoints                                                   | Purpose                                |
|------------|---------------------------------------------------------------------|----------------------------------------|
| Auth       | POST /auth/login · GET /me                                          | Identity + token/session               |
| Repository | GET/POST /documents · GET /documents/:id                            | Metadata, uploads, versions            |
| Search     | GET /search · GET /search/suggest                                   | Hybrid keyword + semantic discovery    |
| GIS        | GET /layers · GET /layers/:id/features · POST /layers               | Map layers and spatial queries         |
| Analytics  | GET /analytics/indicators · POST /analytics/runs                    | Dashboards and analytical jobs         |
| Simulator  | GET /simulations/models · POST /simulations/runs                    | Scenario execution and comparison      |
| Workspaces | GET/POST /workspaces · POST /workspaces/:id/members                 | Collaboration                          |
| Grievances | POST /grievances · GET /grievances · PATCH /grievances/:id/status   | Citizen submission + authorized triage |
| Innovation | GET /challenges · POST /submissions · POST /submissions/:id/reviews | Innovation portal                      |

| Module  | Example endpoints                       | Purpose                          |
|---------|-----------------------------------------|----------------------------------|
| Reports | POST /reports · GET /reports/:id        | Generate/export evidence reports |
| Admin   | GET /admin/users · POST /admin/api-keys | Platform administration          |

Standard response envelope

```
Success
{ data, meta, requestId }
```

```
Error
{ error: { code, message, details },
  requestId }
```

Middleware order: requestId → helmet/CORS → rate limit → authentication → RBAC → validation → controller → service → repository/external adapter → error handler. Validate input with a schema library and never trust client-supplied role/ownership fields.

## 07 Data Ingestion & External API Strategy

Every source is treated as an adapter behind a common ingestion contract.

|          |             |         |         |         |           |
|----------|-------------|---------|---------|---------|-----------|
| 1 Ingest | 2 Normalize | 3 Store | 4 Index | 5 Serve | 6 Explain |
|----------|-------------|---------|---------|---------|-----------|

| Source                                                                                                                                                                                               | Use in prototype                     | Integration pattern                                         |
|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--------------------------------------|-------------------------------------------------------------|
| data.gov.in                                                                                                                                                                                          | Government datasets / indicators     | API adapter → normalize → PostgreSQL → index                |
| Bhuvan / LULC                                                                                                                                                                                        | Land-use / geospatial layers         | API/WMS/download adapter → PostGIS                          |
| OpenStreetMap                                                                                                                                                                                        | Roads/buildings/POIs                 | Overpass adapter → cache → PostGIS                          |
| Copernicus/Sentinel                                                                                                                                                                                  | Satellite imagery / land-use context | Metadata/tiles cached as references — avoid huge raw images |
| OpenAlex / Semantic Scholar                                                                                                                                                                          | Research literature                  | Search adapter → repository metadata + Elasticsearch        |
| NJDG                                                                                                                                                                                                 | Court/dispute context                | External source; mock where unavailable and label clearly   |
| NASA POWER / IMD                                                                                                                                                                                     | Climate context                      | Coordinate/time-series adapter → analytics tables           |
| DIRLMP / SVAMITVA / state records                                                                                                                                                                    | Land/cadastral context               | Use sample datasets if public APIs are unavailable          |
| Provenance is mandatory: every imported dataset/document keeps source name, source URL/reference, retrieval time, dataset/version ID, license note where known, transformation version and checksum. |                                      |                                                             |

## 08 AI Search, RAG & Explainability

AI should enhance discovery and analysis — never become an opaque decision-maker.

### RAG pipeline

- Ingest PDF/document → extract text/OCR if needed → clean → chunk with document/page metadata.
- Generate embeddings → store in the chosen vector/search layer (Elasticsearch if used for vectors, kept as the retrieval index).
- At query time: lexical + semantic retrieval → rank/merge → permission filter → context window.
- Generate the answer with citations pointing back to document ID / page / section and source metadata.
- Log request ID, retrieved document IDs, model/provider version and latency for audit/debugging.

### AI experience by role

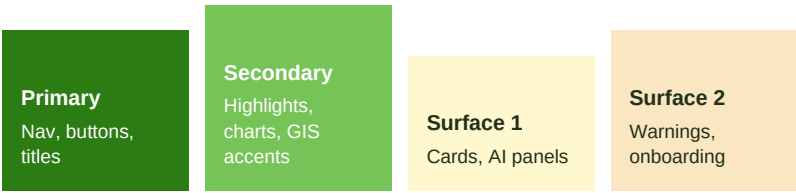
| Role           | AI experience                                                                                    |
|----------------|--------------------------------------------------------------------------------------------------|
| Public         | Ask questions over public resources; summarize public documents; discover related resources      |
| Researcher     | Full repository RAG, literature synthesis, semantic search, recommendations, draft policy briefs |
| Institution    | Organization-level discovery and shared-workspace assistance                                     |
| Polymaker/Govt | Evidence summaries, trend interpretation, scenario comparison, report generation with provenance |
| Admin          | AI configuration/monitoring only — no business-domain decision privileges by default             |

Guardrail: AI outputs must visibly distinguish source-backed facts from generated interpretation. Policy-simulation output is decision support, not an automatic government decision.



## 09 UI System & Stitch MCP Workflow

Use *Stitch MCP* for visual exploration, then make the design system deterministic in code.



| Token      | Value                                                |
|------------|------------------------------------------------------|
| Typography | Noto Sans — bold weight for headings and labels      |
| Radius     | 10–14 px on cards, dialogs, inputs — keep consistent |
| Spacing    | 4 / 8 px rhythm — predictable layout spacing         |

### Core screens to generate in *Stitch MCP*

- Public: landing/search, AI assistant, public repository, GIS viewer, dashboards, grievance form/status.
- Researcher: dashboard, repository upload, document detail, workspace, GIS analysis, AI assistant, innovation submission.
- Institution: org dashboard, researcher roster, shared workspaces, bulk upload, submission oversight.
- Policymaker/Govt: decision dashboard, policy simulator, dispute heatmap/triage, AI report generator, portal admin, audit view.
- Admin: users/roles, API keys, moderation, system health, audit log.

### Stitch → Antigravity handoff loop

Generate screen concept → lock information architecture → extract reusable components → implement in React/Tailwind → connect to Express API → test role visibility → iterate in *Stitch* only for visual improvements, never to redefine business logic.

## 10 Animation & Interaction Blueprint

Motion should make the system feel understandable — short, subtle, accessible.

| Interaction      | Animation                               | Purpose                                 |
|------------------|-----------------------------------------|-----------------------------------------|
| Page transition  | 150–220ms fade/slide                    | Clear navigation without distraction    |
| Search           | Skeleton rows + result fade-in          | Show retrieval is working               |
| AI assistant     | Typing indicator + citation reveal      | Communicate generation and evidence     |
| Dashboard        | Number count-up on load                 | Improve scanability                     |
| GIS              | Layer fade + marker clustering          | Avoid visual jumps when changing layers |
| Policy simulator | Step progress + result reveal           | Make scenario execution understandable  |
| Upload           | Progress bar + success state            | Show ingestion status                   |
| Cards            | Small lift/scale on hover               | Affordance only, no excess motion       |
| Errors           | Shake on invalid field + inline message | Immediate correction cue                |

Accessibility: respect prefers-reduced-motion; never make essential information depend on animation.

## 11 Antigravity Project Structure

*A monorepo so Antigravity can reason about frontend, backend and shared contracts together.*

```
apps/
web/ # React + Vite + Tailwind UI
src/components/ src/pages/ src/features/
src/lib/api/ src/lib/auth/
api/ # Node.js + Express
src/routes/ src/controllers/ src/services/
src/repositories/ src/middleware/ src/adapters/
src/jobs/ src/validators/ src/config/
workers/ # optional analytics/AI job workers
packages/
shared/ # API schemas/types
ui/ # reusable design system
infra/
docker/ migrations/ seed/
docs/
architecture/ api/ prompts/
```

### Backend module rule

- Routes contain HTTP mapping only.
- Controllers translate request → service call, and service result → response.
- Services contain business logic and authorization-aware operations.
- Repositories contain database queries.
- Adapters contain external API integrations and source-specific transforms.
- Jobs handle async ingestion, indexing, embeddings, analytics and report generation.
- Shared schemas define stable request/response contracts.

## 12 Build Sequence for Antigravity

| #  | Phase                  | Exit condition                                                                  |
|----|------------------------|---------------------------------------------------------------------------------|
| 1  | Foundation             | Repo, env config, Docker, lint/test, DB connection, migrations                  |
| 2  | Design system          | Noto Sans, palette, layout, buttons, cards, forms, tables, modal, map shell     |
| 3  | Auth/RBAC              | Login, roles, protected routes, permission middleware, seed users               |
| 4  | Repository             | Upload metadata, object storage abstraction, document detail, tags, permissions |
| 5  | Search/RAG             | Index pipeline, Elasticsearch, embeddings, hybrid search, citations             |
| 6  | GIS                    | PostGIS schema, layer API, map viewer, GeoJSON, filters                         |
| 7  | Dashboards             | Indicators, charts, filters, saved views, role-specific dashboards              |
| 8  | Workspace              | Projects, members, notes, saved resources                                       |
| 9  | Simulator              | Scenario input, job execution, results, explainability, compare scenarios       |
| 10 | Innovation + grievance | Submission/review workflow + geo-tagged grievance/status/triage                 |
| 11 | Integrations           | Connect highest-value external sources; mock unavailable ones transparently     |
| 12 | Hardening              | Security, performance, accessibility, audit trail, demo seed data, e2e tests    |

For the hackathon: don't attempt every external integration before the core journey works. Aim for a working prototype around Bhuvan, data.gov.in and OpenStreetMap, with the repository mocked/simulated where needed.

## 13 Golden Demo Journey

*One continuous evidence-to-decision workflow — proves this is more than a document repository.*

| # | Actor       | Demo action                                  | System response                              |
|---|-------------|----------------------------------------------|----------------------------------------------|
| 1 | Public      | Search “land-use change / dispute / climate” | AI + semantic results with filters           |
| 2 | Researcher  | Open a document, add it to a workspace       | Permissioned workspace update                |
| 3 | Researcher  | Open GIS, select a land-use layer            | PostGIS-backed map visualization             |
| 4 | Researcher  | Ask AI for a synthesis                       | RAG answer with citations/provenance         |
| 5 | Policymaker | Open decision dashboard                      | Indicators + trend charts + dispute heatmap  |
| 6 | Policymaker | Run a what-if policy scenario                | Scenario result + explainable assumptions    |
| 7 | Policymaker | Generate evidence report                     | Report with sources, metrics, audit metadata |
| 8 | Admin       | Open audit view                              | Trace user/action/request/resource           |

## 14 Security, Quality & Acceptance Criteria

- HTTPS everywhere; secure cookie/session strategy if cookies are used; never expose secrets to the frontend.
- RBAC + resource-level authorization on every protected operation.
- Rate-limit authentication, search, AI and public API endpoints.
- Validate file type, size, checksum, and scan uploads before indexing.
- Secrets in an environment/secret manager — never committed API keys.
- Audit events for login, upload, publish, role changes, simulation runs, report generation, admin actions.
- Parameterized SQL/ORM queries and strict input validation.
- Narrow CORS; configured security headers; generic auth errors where account existence is sensitive.
- Keep external-source provenance and retrieval timestamps.

### Definition of done

| Area       | Acceptance check                                                                                         |
|------------|----------------------------------------------------------------------------------------------------------|
| RBAC       | Each role sees only permitted navigation/API data; backend denies forbidden calls even if UI is bypassed |
| Search     | Keyword + semantic query returns ranked results with filters and citations                               |
| Repository | Upload → metadata → storage → index → search is traceable by document ID                                 |
| GIS        | Map supports at least one real/sourced layer and one PostGIS spatial query                               |
| Analytics  | Dashboard metrics traceable to a dataset/version and computation                                         |
| Simulator  | Inputs, model version, run timestamp and outputs stored and explainable                                  |
| AI         | Every answer can expose supporting resources; hallucinated sources are not accepted                      |
| Audit      | Sensitive actions produce audit events with a request ID                                                 |
| UX         | Responsive, Noto Sans, requested palette, reduced-motion support, consistent states                      |

## 15 Antigravity Master Prompt

*Paste or adapt this as the high-level build instruction before delegating modules.*

You are the lead full-stack engineer building SIH 26019 / PS-019: a National Digital Platform for Research, Policy Innovation and Evidence-Based Land Governance. Build a production-structured hackathon MVP.

Preserve the source architecture: Client -> API Gateway -> Auth/RBAC -> Core Services (Search, GIS, Analytics, Policy Simulator, Repository) -> Internal Data / External APIs -> Response Aggregator -> Client.

Constraints:

- Backend: Node.js + Express.js REST APIs.
- Primary database: PostgreSQL + PostGIS. Do not introduce MongoDB for the MVP.
- Search: Elasticsearch.
- Documents: object-storage abstraction + PostgreSQL metadata.
- Cache/jobs: Redis + BullMQ; keep heavy analytics/ingestion workers separable.
- UI: React + Vite + Tailwind CSS as the implementation choice.
- Use Stitch MCP to explore/generate/refine UI screens, then implement reusable components in the codebase.
- Font: Noto Sans. Palette: #2A7C13, #76C457, #FFF8CF, #FBE6C2.
- Use subtle, accessible animations and support reduced motion.
- Implement 5 roles: Public/Citizen, Researcher, Institution, Policymaker/Government Official, Admin. GIS experts belong to Researcher; NGOs may be represented as elevated public users.
- Every protected backend operation must enforce authorization independently of frontend visibility.
- Include provenance, citations and auditability in repository, GIS, analytics, AI and reporting flows.

Build incrementally. Before each major module, inspect existing code, reuse components, avoid duplicate abstractions, and keep API contracts stable. Prefer a modular monolith for the MVP. Seed realistic demo data. Mock an external integration only when live access is unavailable, and label it clearly in the UI.

### Module prompt pattern

Implement <module>.

- Define user stories and RBAC permissions.
- Define PostgreSQL/PostGIS tables and migration.
- Define Express routes, validation schemas and response contracts.
- Implement repository/service/controller/adapters.
- Add a background job if the operation is expensive.
- Build Stitch MCP screen(s), then implement responsive React components.
- Add loading, empty, error and success states.
- Add audit/provenance where applicable.
- Add unit + integration tests.
- Add seed data and a short demo path. Do not change unrelated modules.

## 16 Final Build Checklist

- ☐ Architecture matches the supplied request flow and response-aggregation pattern.
- ☐ PostgreSQL/PostGIS is the authoritative store; Elasticsearch is a derived search index.
- ☐ All five roles have distinct navigation and backend permissions.
- ☐ Repository supports documents, metadata, provenance, versions and search.
- ☐ AI assistant performs permission-aware RAG and exposes citations.
- ☐ GIS viewer reads PostGIS/GeoJSON and at least one external/sourced layer.
- ☐ Analytics dashboard has traceable indicators and demo data.
- ☐ Policy simulator stores scenarios and outputs with explainability metadata.
- ☐ Innovation portal supports challenge/submission/review workflow.
- ☐ Grievance tracker supports geo-tagged submissions and authorized triage.
- ☐ External API adapters are isolated, rate-limited and cached.
- ☐ UI follows Noto Sans and the four supplied colors.
- ☐ Animations are subtle and disabled/reduced under prefers-reduced-motion.
- ☐ Stitch MCP is used for visual exploration, not as the source of business logic.
- ☐ API errors, request IDs and audit events are standardized.
- ☐ Demo runs end-to-end with seeded data even if some external APIs are unavailable.

Source basis: grounded in the supplied SIH 26019 problem statement, the PS-019 request-flow diagram, the API/dataset reference image, and the role/UI notes (26019.pdf, PS019\_Request\_Flow.pdf, plan.png, tools.png, User interface needed.png).